Inteligencia Artificial. Tarea 4

Tomas Santana

Noviembre, 2026

La tarea se realiza en parejas. Todos los modelos se implementan en PyTorch, definiendo cada red como una clase que extiende `nn.Module`.

1. Convolución a mano (4 puntos). Considere la imagen X de 5×5 y el filtro K de 3×3:

$$X = \begin{pmatrix} 1 & 0 & 2 & 1 & 0 \\ 0 & 1 & 3 & 1 & 1 \\ 2 & 1 & 0 & 0 & 2 \\ 1 & 0 & 1 & 2 & 1 \\ 0 & 2 & 1 & 1 & 0 \end{pmatrix}, \qquad K = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$$

   - Calcular a mano la salida de la convolución con stride 1, sin padding y sin sesgo.
   - Aplicar ReLU y después max pooling de 2×2 con stride 1 sobre el resultado.
   - Verificar ambos resultados con `nn.Conv2d` y `nn.MaxPool2d`, asignando a mano los pesos del filtro.
   - ¿Cuántos parámetros tiene una capa `nn.Conv2d(3, 16, kernel_size=3)`? ¿Cuántos tendría una capa densa que recibe una imagen de $32 \times 32 \times 3$ y produce una salida de $30 \times 30 \times 16$?

2. Datos y MLP (4 puntos). Escoger un dataset de imágenes con al menos 3 clases, distinto de MNIST. Por ejemplo, `FashionMNIST`, `KMNIST` o `CIFAR10` de `torchvision.datasets`, o un dataset de Kaggle con imágenes redimensionadas a 64×64 píxeles o menos. Si el dataset tiene más de 20 000 imágenes de entrenamiento, pueden tomar una muestra aleatoria.
   - Mostrar algunas imágenes de cada clase y la proporción de ejemplos por clase.
   - Separar conjuntos de entrenamiento, validación y prueba. Usar la misma separación en todos los modelos de la tarea.
   - Entrenar una red totalmente conectada (`class MLP(nn.Module)`) y graficar la pérdida y la exactitud de entrenamiento y de validación por época.

3. CNN y arquitectura (8 puntos). Sobre los mismos datos:
   - Implementar una red convolucional (`class CNN(nn.Module)`) con al menos 2 bloques de convolución, ReLU y pooling, seguidos de capas densas.
   - Presentar una tabla con la forma de la salida y el número de parámetros de cada capa. Verificar los números con un forward de prueba y con `p.numel()`.
   - Experimentar con la arquitectura: entrenar al menos 4 variantes cambiando, por ejemplo, el número de bloques, el número de filtros, el tamaño del filtro o el tamaño de las capas densas. Presentar una tabla con el número de parámetros y la exactitud de entrenamiento y de validación de cada variante, y explicar qué cambios ayudaron y cuáles no.
   - Mostrar en las curvas de entrenamiento y validación si la mejor arquitectura sobreajusta, y corregirlo con dropout, `weight_decay` o early stopping.
   - Reportar la exactitud de prueba del MLP y de la mejor CNN una sola vez, al final.

4. Análisis (4 puntos).

- Mostrar la matriz de confusión de la mejor CNN sobre el conjunto de prueba, y algunas imágenes mal clasificadas con la probabilidad que les asigna la red.
- Visualizar los filtros de la primera capa convolucional y explicar qué patrones parecen detectar.
- Comparar el número de parámetros y la exactitud del MLP y de la CNN, y explicar por qué la CNN funciona mejor con imágenes.

Entrega y evaluación

Cada pareja debe entregar un notebook (`.ipynb`) con la solución de todos los puntos y el dataset utilizado (o el código que lo descarga). La entrega es **en clase**. El notebook debe ejecutarse de principio a fin sin errores. Deben fijar una semilla y guardar los pesos del modelo final con `torch.save(modelo.state_dict(), ...)`, para poder cargarlo durante la evaluación sin volver a entrenar. La nota definitiva se calculará como:

$$\text{Nota} = \text{Implementación} \times \text{Explicabilidad},$$

donde la implementación es la suma de los puntos anteriores (de 0 a 20) y la explicabilidad es un factor entre 0 y 1 que mide la capacidad de presentar y justificar la solución propia. Para medirla, cada pareja responderá 3 preguntas sobre su implementación, con su código abierto y ejecutable. Cualquiera de los dos integrantes puede ser llamado a responder. El factor se asigna según la siguiente tabla:

Factor	Criterio
1.0	Responde correctamente las 3 preguntas y ubica en su código lo que se le pregunta sin dudar.
0.8	Responde 2 preguntas de forma completa y 1 de forma parcial.
0.6	Responde correctamente 2 preguntas.
0.3	Responde correctamente 1 pregunta.
0	No responde correctamente ninguna pregunta, o no logra ubicar en su propio código lo que se le pregunta.

Una respuesta es **correcta** si contiene todos los puntos esperados y **parcial** si contiene la idea principal pero le falta alguno de ellos.

Temas para preparar la evaluación

Las preguntas se escogerán de un banco que cubre los siguientes temas. Para cada uno, deben poder explicarlo con sus propias palabras y señalar dónde aparece en su código.

1. Calcular una convolución y un max pooling a mano para cualquier imagen y filtro, no solo los de la tarea.
2. La forma de la salida y el número de parámetros de cualquier capa de su red, y por qué la primera capa densa tiene ese número de entradas.
3. Por qué una capa convolucional tiene muchos menos parámetros que una capa densa (pesos compartidos y conexiones locales).
4. Su ciclo de entrenamiento: qué hace cada línea, qué pasa si se omite `zero_grad`, y por qué no se aplica softmax antes de `nn.CrossEntropyLoss`.
5. Qué es una época, un mini-batch y un paso, y cuántos pasos dio su entrenamiento.
6. Sus experimentos de arquitectura: qué cambiaron, qué efecto tuvo cada cambio y por qué escogieron la arquitectura final.
7. Cómo se ve el sobreajuste en sus curvas, qué hace la técnica que usaron para corregirlo, y la diferencia entre `model.train()` y `model.eval()`.
8. Leer su matriz de confusión e interpretar sus filtros: qué clases confunde la red y qué patrones detectan los filtros de la primera capa.